



Computación Paralela



Computación Paralela

- División de tareas de cómputo
 - Puede encontrarse en diferentes niveles
-
- ☐ Pipeline
 - ☐ Threads
 - ☐ Multi-Cores
 - ☐ APIs (Ej MPI - Message Passing Interface)
 - ☐ Clusters - Grids

¿Que es un hilo/thread ?

- Parte de un programa que se puede ejecutar de manera independiente de otras partes de código.
- Se implementan **junto al Sistema Operativo** con una librería
- Un proceso puede tener por ejemplo 2 hilos.
- Corren dentro del mismo núcleo (core)
- Al compartir recursos (ej. Memoria) se debe cuidar la sincronización (ej. Exclusión tipo Mutex)



Hyper-Threading

- Tecnología desarrollada por Intel en 2002
- Permite que un núcleo físico de procesador funcione como dos núcleos lógicos desde el SO
- Puede aumentar el rendimiento entre 15-30%
- No duplica el rendimiento
- AMD tiene una tecnología equivalente llamada **SMT** (Simultaneous Multithreading)

Ejemplo Intel

- Procesador Intel® Core™ i9-10900K
- Cantidad de núcleos : 10
- Cantidad de subprocesos: 20

Tecnología Intel® Hyper-Threading ‡

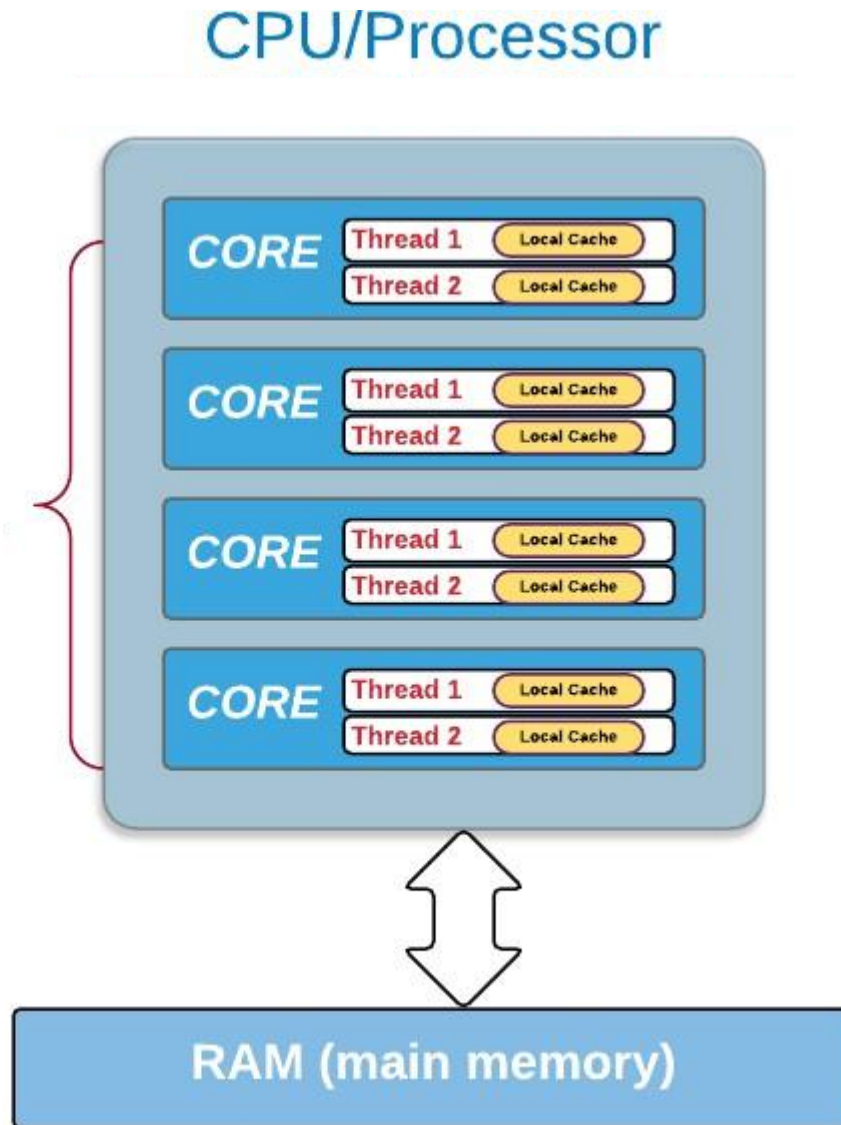


La Tecnología Intel® Hyper-Threading ofrece dos cadenas de procesamiento por núcleo físico. Las aplicaciones con muchos subprocesos pueden realizar más trabajo en paralelo, completando antes las tareas.

 Encuentre productos con Tecnología Intel® Hyper-Threading ‡

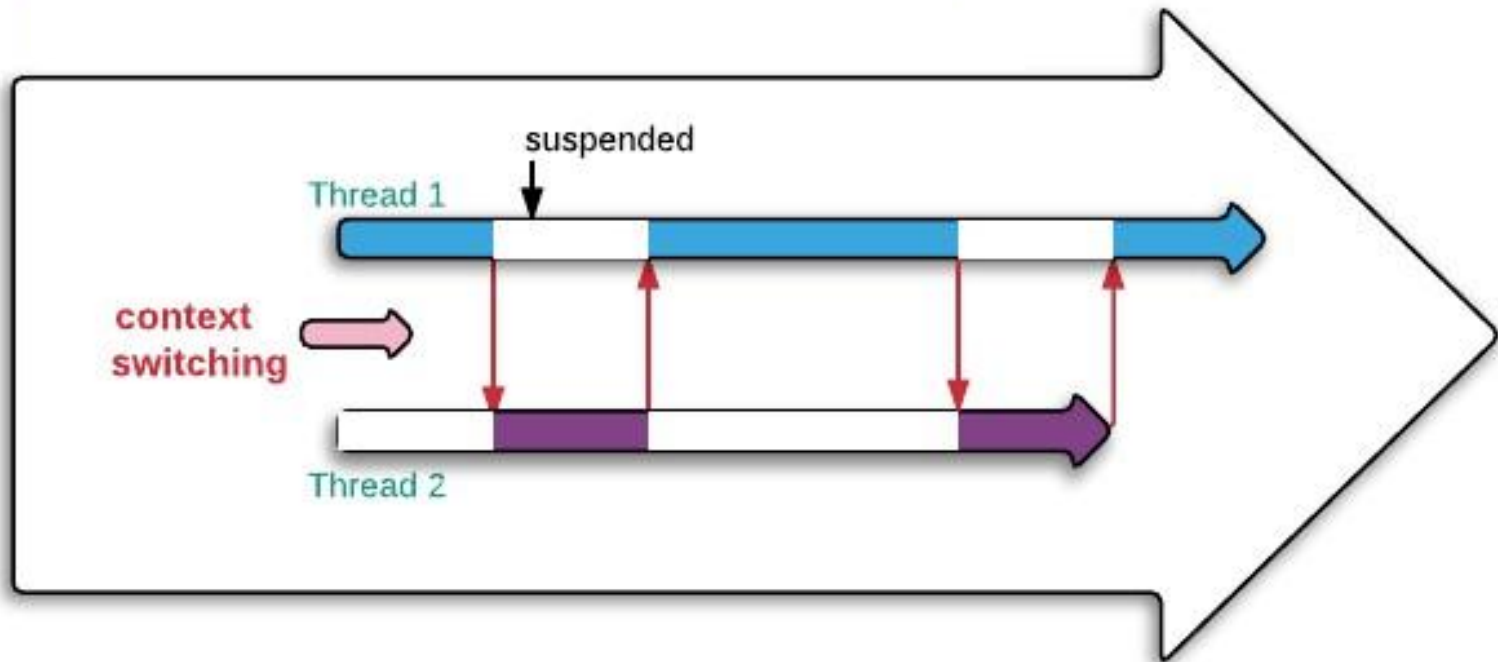
¿Que es un hilo/thread ?

Si el CPU soporta Hyper-Threading, por ejemplo podemos tener dos hilos por core, sino es un solo hilo por core.



¿Que es un hilo/thread ?

Ejemplo de un proceso corriendo en un core con dos hilos



Aparece el concepto de Mutex (Mutual Exclusion) para poder usar mismos recursos (ej. dato en RAM) por ambos hilos

Ejemplo en C

Usando la librería **pthread** (POSIX threads)

Algunas funciones:

<code>pthread_create()</code>	Crea un nuevo thread
<code>pthread_join()</code>	Espera a que un thread termine
<code>pthread_self()</code>	Obtiene el ID del thread actual

Se compila con: `gcc -o threads ejemplo.c -pthread`

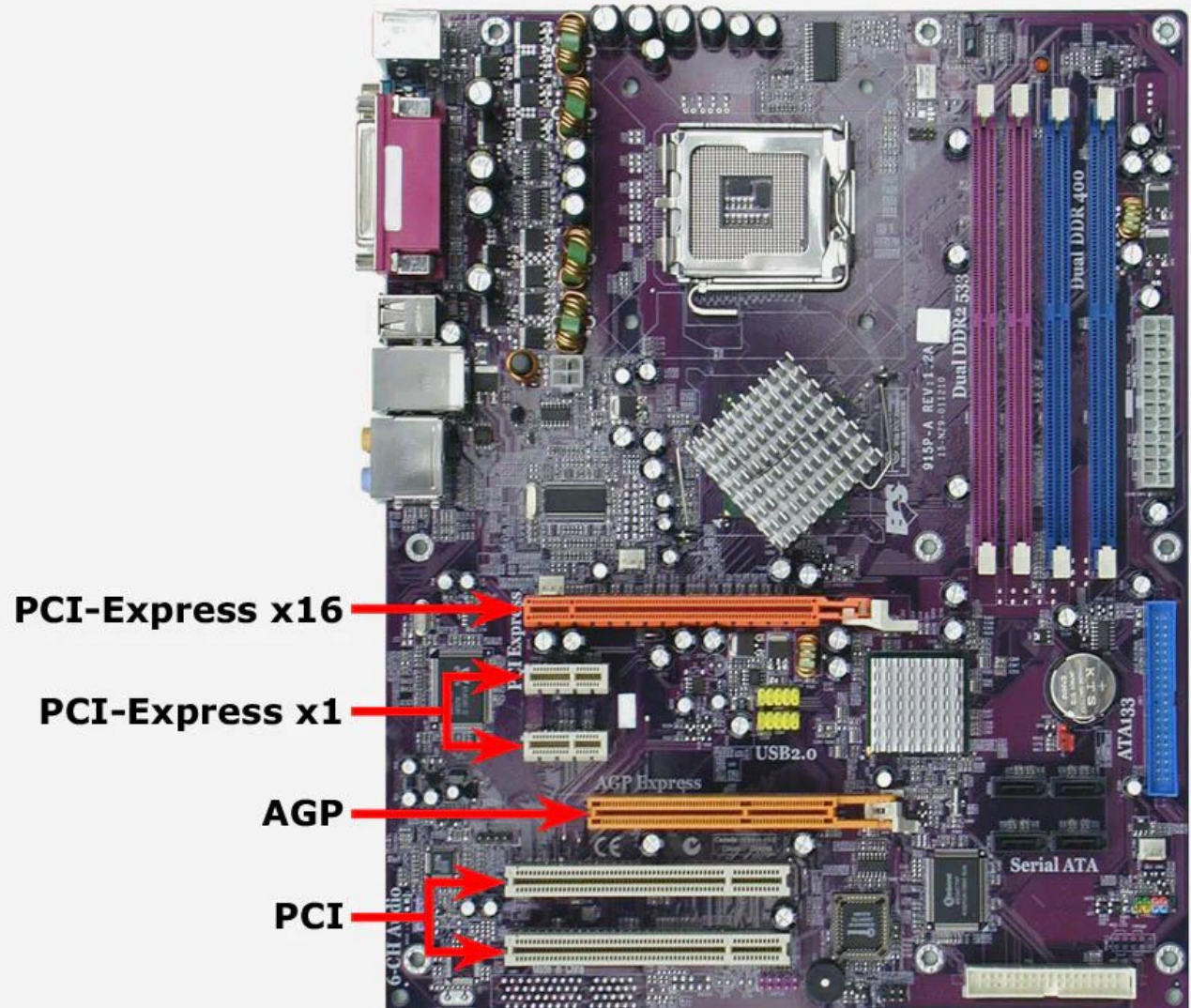
[Ejemplo Completo](#)



GPU

¿Dónde?

- En una PC la GPU se conecta a un Bus
- Hay varios tipos de Buses según velocidad, y consumo
- Según el modelo de GPU es el BUS que usa



Externas

- Con puertos especiales (Thunderbolt)
- RIGs de minería



Momentos de fama

- Video juegos
- Criptomonedas
- IA



3D Voodoo Quake - 1996



GPU

- Diseñada para ejecutar miles de threads en paralelo
- **Menor velocidad de ejecución de UN thread que una CPU**
- Pero más cantidad.

Fabricantes de GPU

- NVIDIA (tomaremos esta marca de referencia)
- AMD
- ATI
-





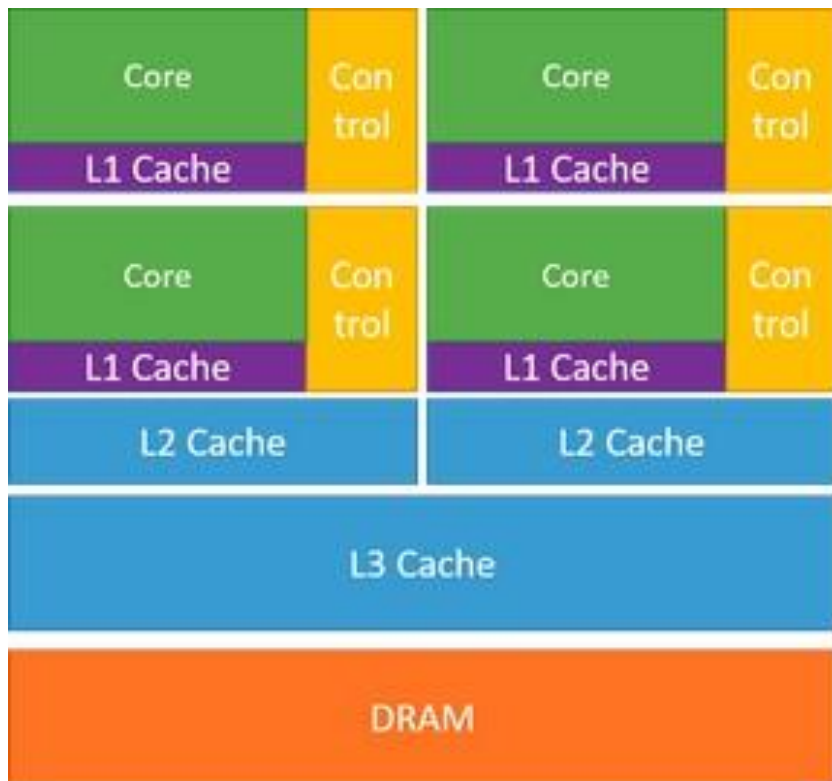
Optimización

- En **CPU**
- Se busca optimizar código **secuencial**
- En **GPU**
- Se busca optimizar código en **paralelo**
- GPU tiene 5 ó 10 veces más de bus a la memoria

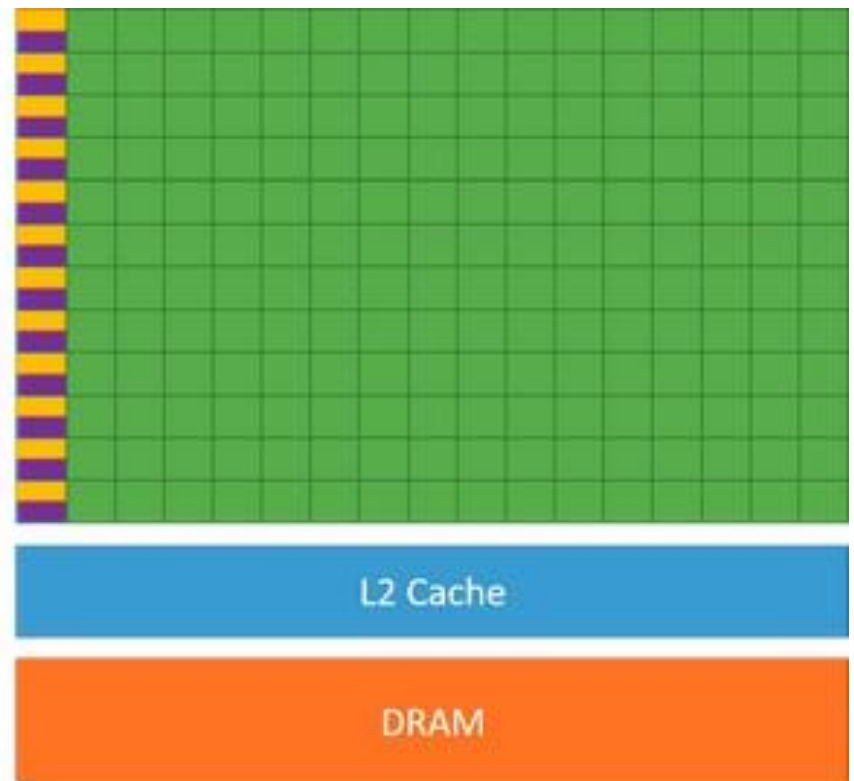
Entonces se usan ambas y se denomina:

Computacion paralela heterogenea

GPU - Arquitectura minima



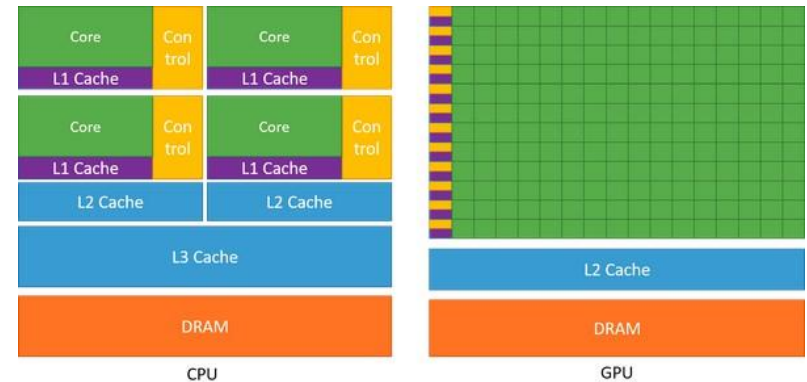
CPU



GPU

GPU - Arquitectura minima

- En **CPU**
- Se busca minimizar latencia
- En **GPU**
- Se busca mejorar cantidad de datos y rendimiento (throughput)



3 maneras de acelerar apps

Applications

Libraries

Easy to use
Most Performance

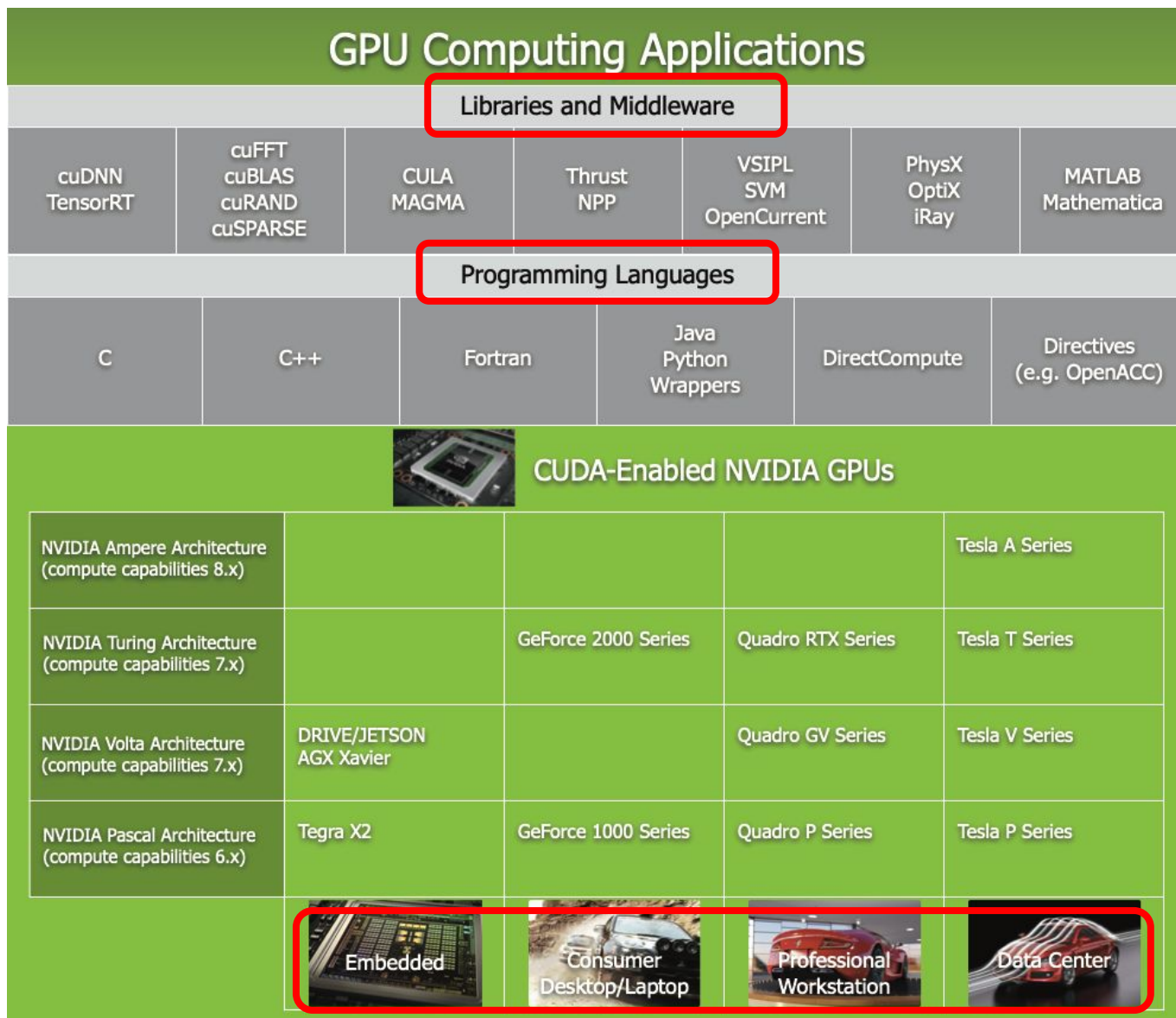
Compiler
Directives

Easy to use
Portable code

Programming
Languages

Most Performance
Most Flexibility

Muchas combinaciones





CUDA



Nvidia - CUDA

- Compute Unified Device Architecture
- Nace en 2006 para facilitar la programación de las GPU
- CUDA Architecture + CUDA Programming Model
- Es un toolkit para programar, extensión de C



CUDA

- Facilita la "computación heterogénea", "mezcla entre computación **paralela** y computación **serial**".
- Tareas de paralelo a la GPU
- Tareas serie a la CPU
- Se programa en C, C++, OpenCL (como ejemplos)

Paralelización

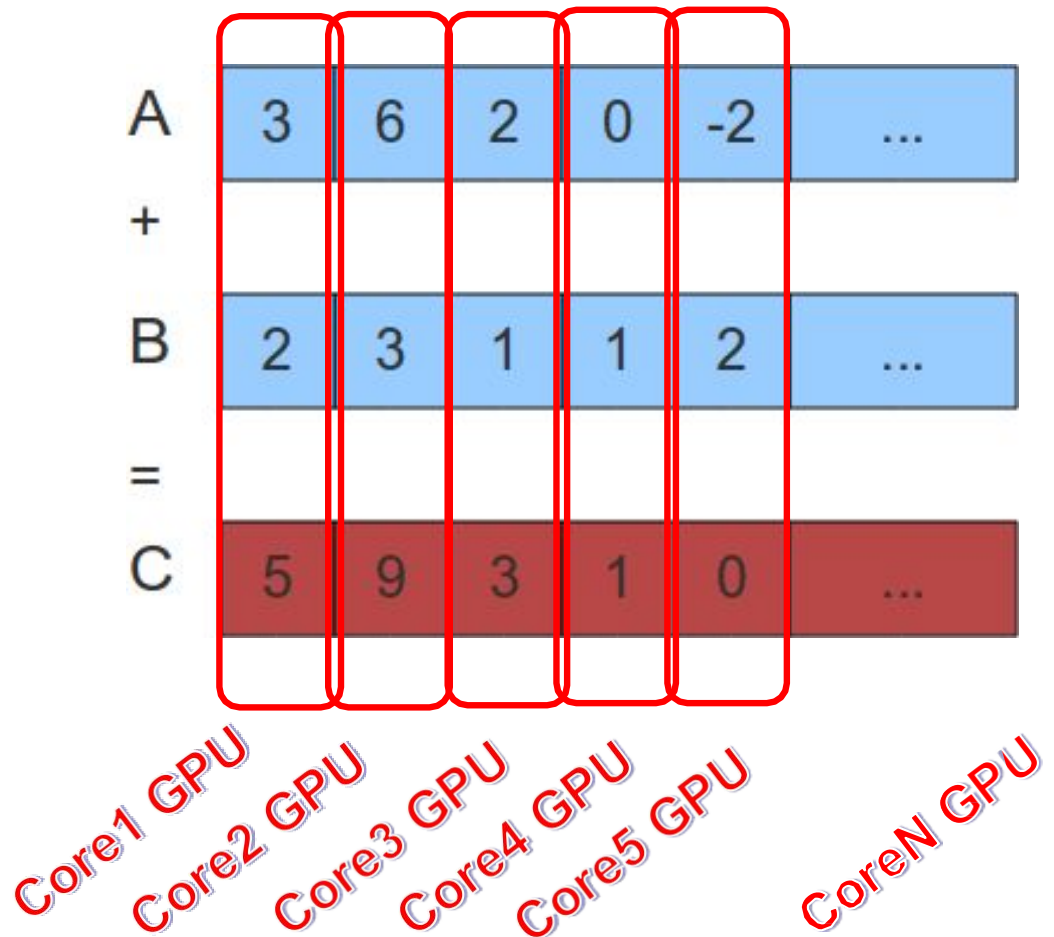
- Ejemplo de suma de dos vectores
- En CPU de un hilo, recorre cada elemento, suma e incrementa, de manera secuencial.
- En CPU de dos hilos se podría paralelizar, sumar elementos a la vez

A	3	6	2	0	-2	...
+						
B	2	3	1	1	2	...
=						
C	5	9	3	1	0	...

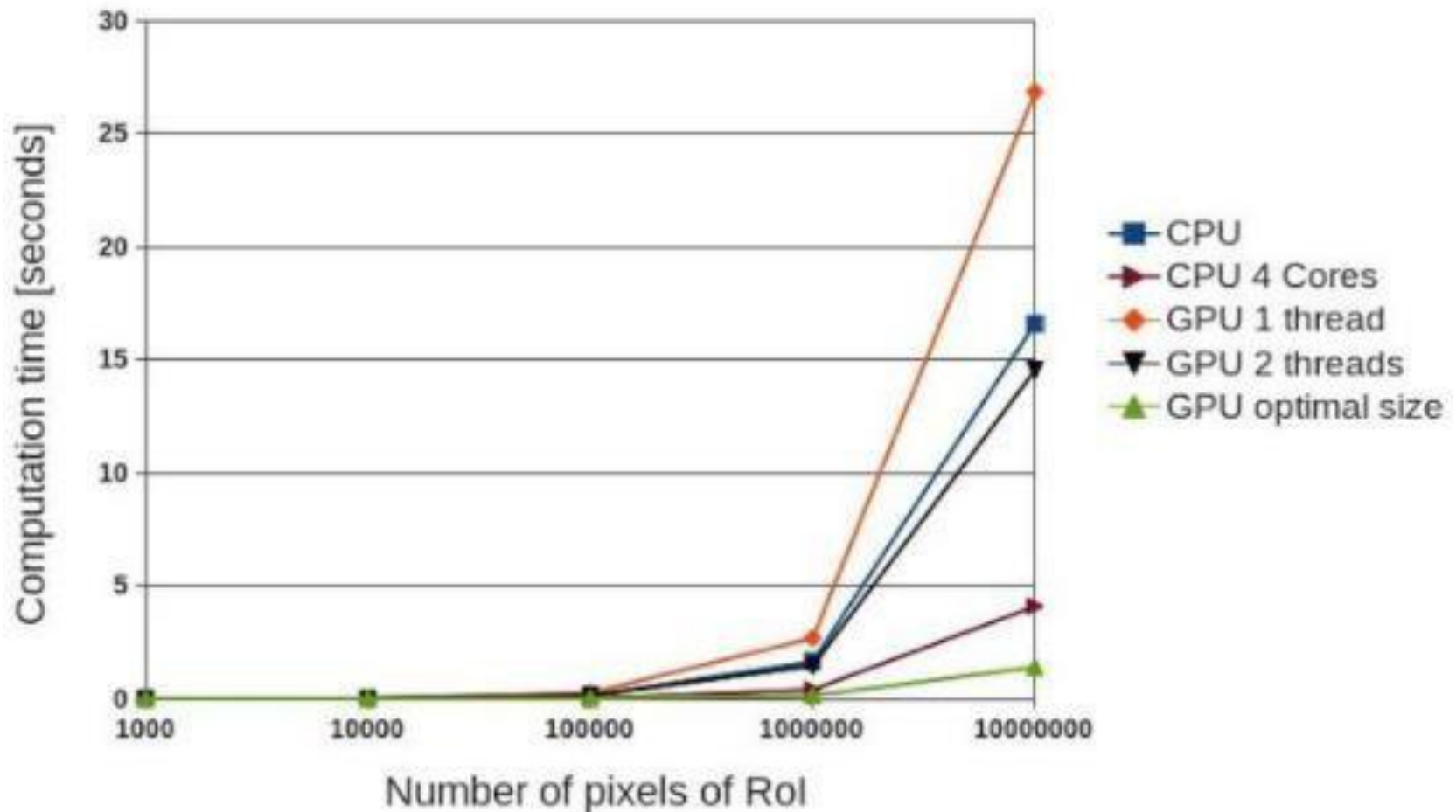
Y con la GPU ?

Paralelización - GPU

- Ejemplo de suma de dos vectores
- Cada suma podría ser ejecutada simultáneamente por un procesador (virtual) de la GPU



Paralelización – GPU vs CPU



Ejemplo de tiempo de respuesta en alteración de pixeles. Observar el rendimiento de un hilo de GPU

Pasos para ejecutar código

Definiciones

“Host”: La CPU del sistema.

“Device”: La GPU

Para ejecutar un programa de CUDA

- 1) Se copia los datos de entrada de host-memory a device-memory
- 2) Se carga el programa en device-memory y se ejecuta
- 3) La salida del programa se copia de device-memory a host-memory

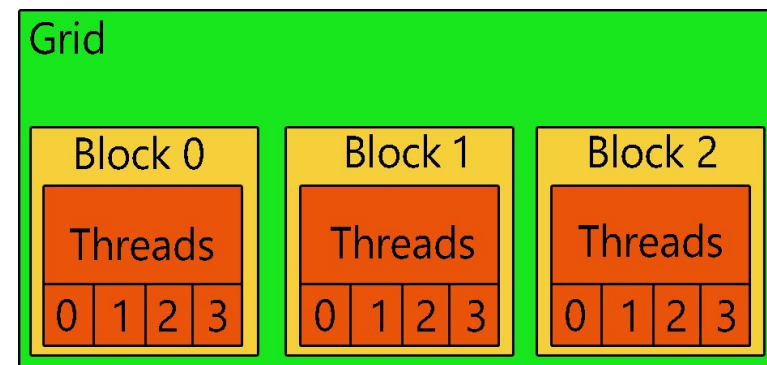
Hilos, Grids y Blocks

- **Hilo (thread)**: la unidad mínima de ejecución en la GPU.
 - Ejecuta instrucciones de un kernel sobre un dato (por ejemplo, calcula $C[i] = A[i] + B[i]$ para un i distinto).
- **Bloque (block)**: un grupo de hilos que pueden comunicarse rápido entre sí y sincronizarse.
 - Tienen **memoria compartida** para sincronizar hilos.
- **Rejilla (grid)**: conjunto de bloques que ejecutan una llamada de kernel.
 - Todos los bloques del grid ejecutan el mismo kernel pero con índices de bloque distintos.

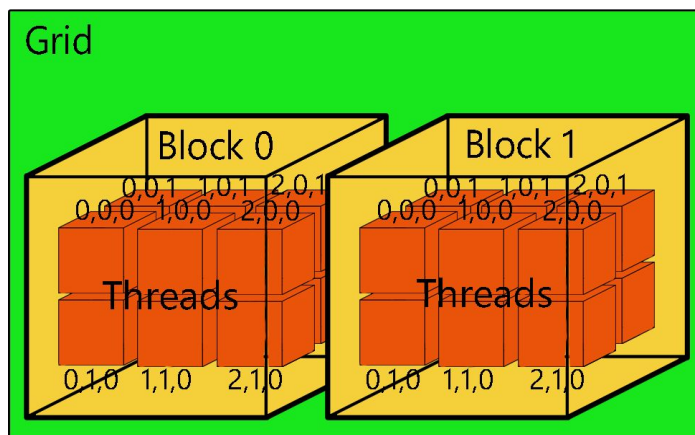
Threads, Grids y Blocks

- Bloques con hilos de una, dos y tres dimensiones
- Cada hilo sabe a qué bloque corresponde y cuántos bloques hay en el grid.

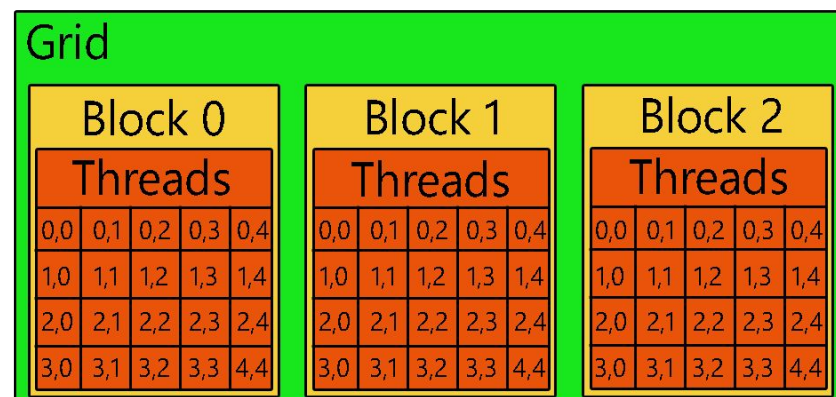
1D Grid of 1D Blocks



1D Grid of 3D Blocks



1D Grid of 2D Blocks



Threads, Grids y Blocks

- En CUDA al programar se debe definir cuántos hilos serán usados en cada bloque
- Para eso existen las variables:

blockIdx : Id de bloque dentro de la grilla

threadIdx: Id de hilo dentro del bloque

blockDim: Numero de hilos por bloque

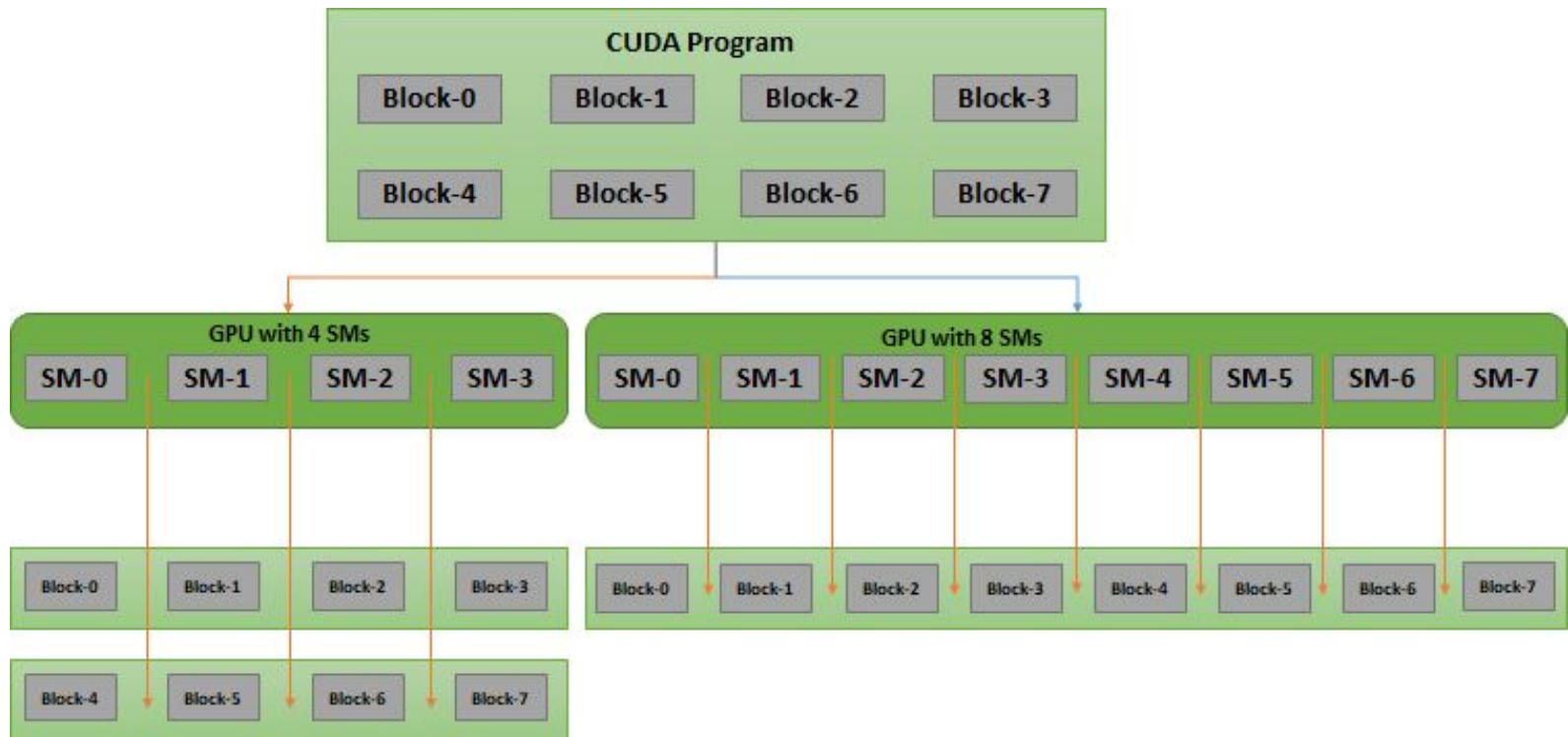
Estas variables tienen un atributo por eje cartesiano x, y, z, por ejemplo para el X

blockIdx.x Id de bloque dentro de la grilla para la dimensión X

threadIdx.x Id de hilos dentro del bloque para la dimensión X

blockDim.x numero de hilos de bloque para la dimensión X

Pasos para ejecutar código



Un programa CUDA al correr en una GPU con 4 SM ó en una GPU con 8 SM. Es transparente para el programador. Lo soluciona CUDA.

Sincronización

Al tener dos procesadores (CPU y GPU) es importante la sincronización

Se puede esperar a que termine la GPU o no.

Función desde CPU que espera que finalice GPU:

`cudaDeviceSynchronize()`

Programación básica de CPU+GPU

- La CPU aún controla el flujo de trabajo
- Veamos este ejemplo donde se declara un función de tipo `__global__` para correr en GPU
- Luego lanza **un bloque** con **un hilo** `<<1,1>>`
 - La CPU continúa trabajando
- Con funcion `cudaDeviceSynchronize()` espera que GPU termine

Tanto CPU cómo GPU imprimen un Hola Mundo

Hola Mundo CPU+GPU

- Vamos a probar la plataforma: <https://leetgpu.com/playground>

```
#include <stdio.h>

// Kernel que se ejecuta en el dispositivo (GPU)
__global__ void holaMundoGPU() {
    printf("¡Hola, mundo desde el GPU!\n");
}

// Función principal que se ejecuta en el host (CPU)
int main() {
    printf("¡Hola, mundo desde el CPU!\n");

    // Lanzar el kernel con 1 bloque de 1 hilo
    holaMundoGPU<<<1, 1>>>();

    // Esperar a que el dispositivo termine antes de salir
    cudaDeviceSynchronize();
    return 0;
}
```

Ejemplo de múltiples hilos

- Veamos este ejemplo donde se declara **un** bloque con 32 hilos y se los llama sólo una vez.

```
#include <stdio.h>

__global__ void hello_kernel()
{
    printf("Hello from GPU thread %d\n", threadIdx.x);
}

int main()
{
    hello_kernel<<<1, 32>>>();

    cudaDeviceSynchronize();

    return 0;
}
```

Ejemplo de múltiples bloques

- La salida son 32 printf con el ID del thread que va de 0 a 31

En leetGPU se debe hacer RUN en modo “Cycle Acurate” para este caso

Console Output

```
Running NVIDIA GTX TITAN X in CYCLE ACCURATE mode...
Compiling...
Executing...
Hello from GPU thread 0
Hello from GPU thread 1
Hello from GPU thread 2
Hello from GPU thread 3
Hello from GPU thread 4
Hello from GPU thread 5
```

```
Hello from GPU thread 28
Hello from GPU thread 29
Hello from GPU thread 30
Hello from GPU thread 31
GPU Execution Time: 3.81 microseconds
Exit status: 0
```



Ejemplo de pasaje de variables

- Veamos este ejemplo donde se declaran variables desde la CPU y se pasan a la GPU para que realice operaciones.
- En este caso es un suma simple que realiza CPU y GPU

[Codigo completo](#)

Funciones en CPU+GPU (1)

```
#include <stdio.h>

// Kernel: suma dos números en la GPU
__global__ void suma(float *a, float *b, float *resultado)
{
    *resultado = *a + *b;
    printf("GPU: Calculando %.0f + %.0f = %.0f\n", *a, *b,
*resultado);
}

int main()
{
    printf("=== TRANSFERENCIA DE DATOS CPU <-> GPU ===\n\n");

    // PASO 1: Crear datos en CPU (host)
    printf("PASO 1: Datos en CPU\n");
    float h_a = 5.0f;
    float h_b = 3.0f;
    float h_resultado = 0.0f;
    printf("CPU: a = %.0f, b = %.0f\n\n", h_a, h_b);
```

Funciones en CPU+GPU (2)

```
// PASO 2: Reservar memoria en GPU (device)
```

```
printf("PASO 2: Reservar memoria en GPU\n");
```

```
float *d_a, *d_b, *d_resultado;
```

```
cudaMalloc(&d_a, sizeof(float));
```

```
cudaMalloc(&d_b, sizeof(float));
```

```
cudaMalloc(&d_resultado, sizeof(float));
```

```
printf("GPU: Memoria reservada ✓\n\n");
```

```
// PASO 3: Copiar datos de CPU a GPU
```

```
printf("PASO 3: CPU → GPU (cudaMemcpy)\n");
```

```
cudaMemcpy(d_a, &h_a, sizeof(float), cudaMemcpyHostToDevice);
```

```
cudaMemcpy(d_b, &h_b, sizeof(float), cudaMemcpyHostToDevice);
```

```
printf("GPU: Datos copiados → a=%.0f, b=%.0f\n\n", h_a, h_b);
```

```
// PASO 4: Ejecutar kernel en GPU
```

```
printf("PASO 4: Ejecutar kernel en GPU\n");
```

```
suma<<<1, 1>>>(d_a, d_b, d_resultado);
```

```
cudaDeviceSynchronize();
```

```
printf("\n");
```

Funciones en CPU+GPU (3)

```
// PASO 5: Copiar resultado de GPU a CPU
printf("PASO 5: GPU → CPU (cudaMemcpy)\n");
cudaMemcpy(&h_resultado, d_resultado, sizeof(float),
cudaMemcpyDeviceToHost);
printf("CPU: Resultado recibido ✓\n\n");

// PASO 6: Mostrar resultado en CPU
printf("PASO 6: Resultado final en CPU\n");
printf("CPU: %.0f + %.0f = %.0f\n\n", h_a, h_b,
h_resultado);

// PASO 7: Liberar memoria GPU
printf("PASO 7: Liberar memoria GPU\n");
cudaFree(d_a);
cudaFree(d_b);
cudaFree(d_resultado);
printf("GPU: Memoria liberada ✓\n");

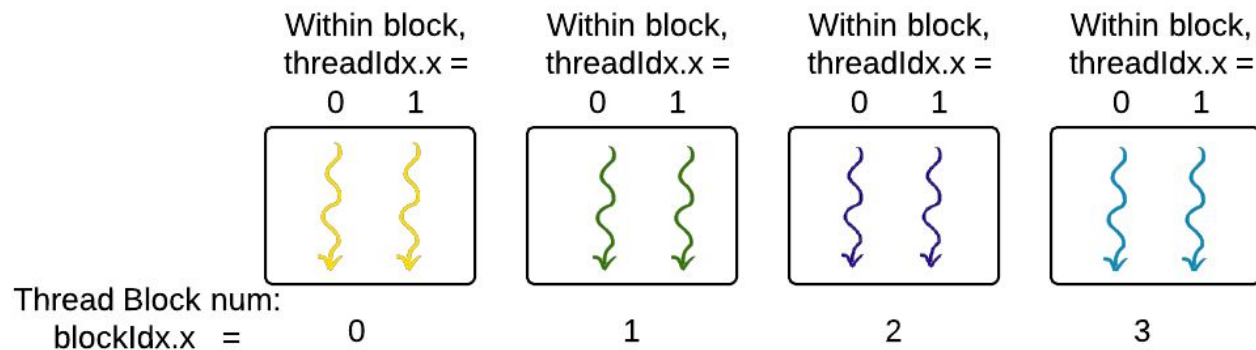
return 0;
}
```

Ejemplo teórico con bloques

- Queremos sumar 2 vectores de 8 elementos usando 8 hilos.
- Una posible organización son 4 bloques de dos hilos cada uno.
- Como es un problema de una sola dimensión, usamos la dimensión X
- Para determinar el ID de cada hilo usaremos la ecuación:
- $\text{int tid} = \text{blockDim.x} * \text{blockIdx.x} + \text{threadIdx.x};$

Ejemplo

1- dimensional Grid of Blocks of Threads,
where number of threads per block,
 $\text{blockDim.x} = 2$



Processing Unit Assignment

	0		1		2		3	
index	0	1	2	3	4	5	6	7
Array A								
B								
C								



Ejemplos Prácticos (Colab)



Google Colab

- Veamos de que se trata

<https://colab.research.google.com/notebooks/intro.ipynb>

- Cómo podemos usarlo para lenguaje C

[Google Colab para compilar](#)

Colab y Terminal

- Que procesador y sistema operativo nos dan

Comandos para analizar Colab

- Autocompleta con IA desde comentarios

```
[ ] #Dirección IP  
!hostname -I
```

```
#Consulta para saber la IP publica con la que salgo a Internet desde este host  
!curl ipinfo.io
```

GPU en Colab

- Vamos a conectarnos a una GPU

Cambiar tipo de entorno de ejecución

Tipo de entorno de ejecución

Python 3 ▼

Acelerador de hardware ?

☒ CPU ☐ GPU T4 ☐ GPU A100 ☐ GPU L4

☐ v5e-1 TPU ☐ v6e-1 TPU

¿Quieres acceder a GPU premium?
[Compra unidades de procesamiento adicionales](#)

Versión de entorno de ejecución ?

Más reciente (opción recomendada) ▼

Cancelar Guardar

Análisis de GPU de Colab



TPU

- Tensor Processing Unit
- Acelerador de hardware diseñado por Google específicamente para operaciones de machine learning y deep learning.
- Están optimizadas para operaciones matriciales y tensoriales que son fundamentales en redes neuronales
- Tensores: Los tensores son matrices multidimensionales



Compara velocidades

- Veamos un código que compara:
 - Velocidad de CPU
 - Velocidad de un hilo de GPU
 - Velocidad de varios hilos de GPU

[Comparativa de cálculos simples y complejos](#)

Analogías de funciones

CUDA	Equivalente CPU	Propósito
<code>cudaMallocManaged()</code>	<code>malloc()</code>	Reservar memoria
<code>cudaFree()</code>	<code>free()</code>	Liberar memoria
<code>cudaEventRecord()</code>	<code>clock()</code>	Marcar tiempo
<code>cudaEventElapsedTime()</code>	<code>difftime()</code>	Calcular duración
<code>cudaGetLastError()</code>	<code>errno</code>	Verificar errores
<code><<<bloques, hilos>>></code>	(no existe)	Lanzar kernel paralelo

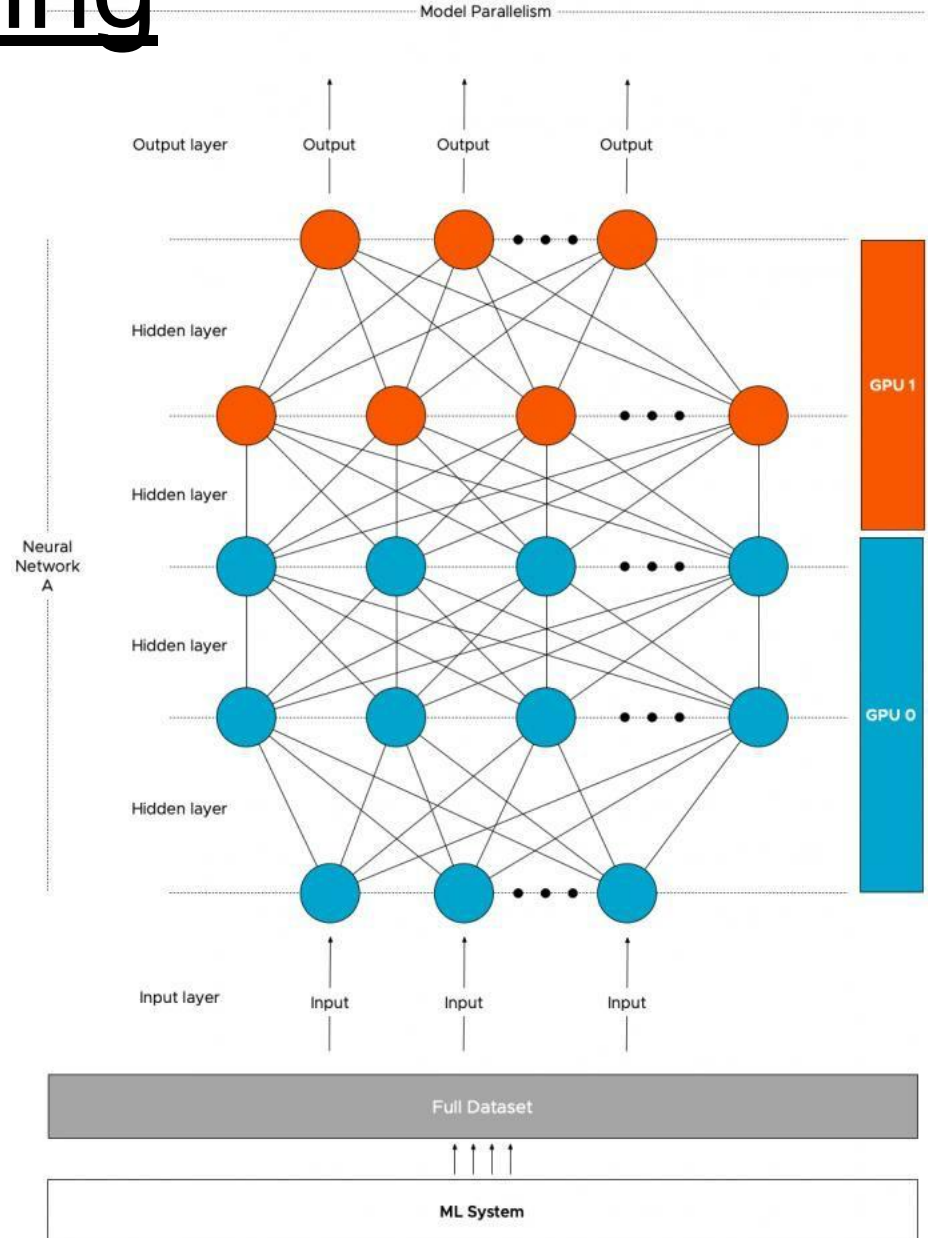


Usos de GPU

- Gráficos
- **Machine Learning**
- Minería de Criptomonedas.
- Password Cracking

Machine Learning

En **redes neuronales** los nodos pueden realizar su tarea computacional de manera paralela.



Ejemplo Titan XP

- Comando deviceQuery

(./cudasamples/samples/bin/x86_64/linux/release/deviceQuery)



- ✓ 12 GB de Memoria
- ✓ 30 Multiprocesadores
- ✓ 128 CUDA cores por Multiprocesador (= 3840 CUDA Cores)
- ✓ 3 MB de Caché L2
- ✓ 2048 threads máximo por Multiprocesador

Cuda Core != Core

Hash rate

- Término tomado de minería de criptomonedas
- La unidad es hash/segundo
- Mide cuantos hashes, según el algoritmo puede generar la placa.
- Hash es una función/algoritmo que se aplica a un texto
- Ejemplos: **MD5("hola") = 4d186321c1a7f0f354b297e8914ab240**
- Se obtiene un “digesto” y tiene varias características (por ejemplo no se debería desde un digesto poder llegar al texto original)

Ejemplo Hashcat en Titán

```
svalles@titan-arq:~$ hashcat -b  
hashcat (v4.0.1) starting in benchmark mode...
```

```
OpenCL Platform #1: NVIDIA Corporation  
=====
```

* Device #1: TITAN Xp, 3048/12194 MB allocatable, 30MCU

```
Benchmark relevant options:  
=====
```

* --optimized-kernel-enable

```
Hashmode: 900 - MD4  
  
Speed.Dev.#1.....: 52249.2 MH/s (20.47ms)
```

```
Hashmode: 0 - MD5
```

```
Speed.Dev.#1.....: 31457.3 MH/s (31.99ms)
```

```
Hashmode: 100 - SHA1
```

```
Speed.Dev.#1.....: 11864.4 MH/s (85.33ms)
```

Ejemplo Hashcat en i7 con GeForce

```
CUDA API (CUDA 11.1)
=====
* Device #1: GeForce 940MX, 1689/2048 MB, 3MCU

OpenCL API (OpenCL 1.2 CUDA 11.1.114) - Platform #1 [NVIDIA Corporation]
=====
* Device #2: GeForce 940MX, skipped

OpenCL API (OpenCL 2.1 ) - Platform #2 [Intel(R) Corporation]
=====
* Device #3: Intel(R) HD Graphics 620, 6260/6324 MB (2047 MB allocatable), 24MCU
* Device #4: Intel(R) Core(TM) i7-7500U CPU @ 2.70GHz, skipped

Benchmark relevant options:
=====
* --optimized-kernel-enable

Hashmode: 0 - MD5

Speed.#1.....: 2350.7 MH/s (85.07ms) @ Accel:64 Loops:1024 Thr:1024 Vec:8
Speed.#3.....: 507.6 MH/s (48.06ms) @ Accel:512 Loops:256 Thr:8 Vec:4
Speed.#*.....: 2858.3 MH/s

Hashmode: 100 - SHA1

Speed.#1.....: 782.0 MH/s (63.74ms) @ Accel:16 Loops:1024 Thr:1024 Vec:1
Speed.#3.....: 61136.0 kH/s (48.85ms) @ Accel:32 Loops:512 Thr:8 Vec:4
Speed.#*.....: 843.2 MH/s

Hashmode: 1400 - SHA2-256

Speed.#1.....: 279.7 MH/s (89.49ms) @ Accel:8 Loops:1024 Thr:1024 Vec:1
Speed.#3.....: 67488.3 kH/s (92.23ms) @ Accel:512 Loops:64 Thr:8 Vec:4
Speed.#*.....: 347.2 MH/s

Hashmode: 1700 - SHA2-512
```

Resultados en MH/s

Placa	MD5	SHA1	SHA256
Titan XP	31457	11864	4407
GeForce 940MX	2350	782	279
HD Graphics 620	507	61	64
i7-7500U	226	132	53

